

Online Judge V1: High-Level Design (HLD)

0. High-Level Architecture Diagram

[Architecture Diagram Pending]

A high-level system architecture diagram illustrating the data flow between the React Frontend, Node.js/Express Backend, PostgreSQL Database, and the BullMQ/Docker Execution Pipeline is currently being drafted and will be inserted here shortly.

1. Tech Stack

- **Frontend:** React + Redux (State Management).
- **Backend:** Node.js + Express.js.
- **Database:** PostgreSQL (Relational schema enforcing ACID compliance).
- **Message Broker:** Redis + BullMQ (For asynchronous job queuing).
- **Execution Sandbox:** Docker (Single container instantiation per submission).

2. Database Schema (PostgreSQL)

2.1. Tables

2.1.1. Users Table

Handles authentication, user profiles, and role-based access (RBAC).

Column Name	Data Type	Constraints & Keys
user_id	UUID / SERIAL	Primary Key
username	VARCHAR	Unique, Not Null
email_id	VARCHAR	Unique, Not Null

Column Name	Data Type	Constraints & Keys
hashed_password	VARCHAR	Not Null
role	VARCHAR	Enum: 'user', 'admin' (Default: 'user')
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP

2.1.2. Problems Table

Stores the core problem data and UI-facing sample cases.

Column Name	Data Type	Constraints & Keys
problem_id	UUID / SERIAL	Primary Key
problem_name	VARCHAR	Not Null
difficulty	VARCHAR	Enum: Easy, Medium, Hard
statement	TEXT	Not Null
sample_testcases	JSONB	Not Null

2.1.3. Test Cases Table

Stores the hidden test cases for the execution engine.

Column Name	Data Type	Constraints & Keys
testcase_id	UUID / SERIAL	Primary Key

Column Name	Data Type	Constraints & Keys
problem_id	UUID / INT	Foreign Key - Problems(problem_id)
input	TEXT	Not Null
output	TEXT	Not Null

2.1.4. Submissions Table

Tracks all code executions, historical data, and verdicts.

Column Name	Data Type	Constraints & Keys
submission_id	UUID / SERIAL	Primary Key
user_id	UUID / INT	Foreign Key - Users(user_id)
problem_id	UUID / INT	Foreign Key - Problems(problem_id)
code	TEXT	Not Null
language	VARCHAR	Enum: cpp, python, java
verdict	VARCHAR	Enum: Pending, Accepted, Wrong Answer, Compilation Error, Runtime Error, Time Limit Exceeded, Memory Limit Exceeded
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP

2.2 Entity Relationships

- **Users to Submissions - One-to-many:** A single user can make multiple submissions, but each submission belongs to exactly one user.
- **Problems to Submissions - One-to-many:** A single problem can have multiple submissions across different users.
- **Problems to Test Cases - One-to-many:** A single problem contains multiple hidden test cases.

2.3 Database Indexes

1. **Submissions (user_id, problem_id):** A composite index that heavily optimizes the GET `/api/submissions/problem/:id/me` endpoint to instantly fetch a specific user's history for the exact problem they are currently viewing.
2. **Submissions (user_id):** Optimizes querying the millions of rows in the Submissions table for a single user's entire history, powering the GET `/api/submissions/user/:id/all` endpoint.
3. **Submissions (problem_id):** Optimizes the GET `/api/submissions/problem/:id/all` endpoint to fetch all attempts across the platform for a specific problem.
4. **Submissions (created_at):** Optimizes sorting for recent submissions and leaderboard aggregations.
5. **Test Cases (problem_id):** Ensures the Worker instantly retrieves all hidden test cases when evaluating a fresh submission.

3. Backend REST API Endpoints (Express.js)

Endpoint	HTTP Method	Purpose
<code>/api/auth/register</code>	POST	Create a new user. Return JWT.
<code>/api/auth/login</code>	POST	Authenticate user. Return JWT.

Endpoint	HTTP Method	Purpose
/api/auth/logout	POST	Logs user out, clears HTTP-only cookie.
/api/admin/*	ALL	Protected routes for Admin CRUD operations on problems and testcases.
/api/problems	GET	Fetch problem list for the dashboard.
/api/problems/:id	GET	Fetch specific problem details and sample test cases.
/api/submissions/run	POST	Test code against custom/sample testcases. Returns raw stdout. Does not affect database/leaderboard.
/api/submissions/submit	POST	Official evaluation against hidden test cases. Saves submission and updates leaderboard.
/api/submissions/:id/status	GET	Polling Endpoint. Continuously fetches the current verdict for a specific submission without needing a full page refresh.

Endpoint	HTTP Method	Purpose
/api/submissions/problem/:id/me	GET	Fetch the current logged-in user's submissions for a specific problem. Populates the "My Submissions" tab.
/api/submissions/problem/:id/all	GET	Fetch all users' submissions for a specific problem. Populates the "All Submissions" tab.
/api/submissions/user/:id/all	GET	Fetch the complete submission history for a specific user across every problem they have attempted.
/api/leaderboard	GET	Fetch rankings based on accepted verdicts.

4. Execution Engine & Pipeline

Sequential Evaluation Pipeline (For Submit Route)

6. **Queue Pickup:** BullMQ worker picks up a Pending submission.
7. **Compilation:** Failure results in "Compilation Error" as the verdict.
8. **Boot-up:** A single Docker container is spun up tailored to the submitted language.
9. **Sequential Streaming:** Worker iterates through all hidden test cases for the problem, injecting stdin and awaiting stdout.
 - If memory exceeds 256 MB or crashes: "Memory Limit Exceeded" (MLE).
 - If execution exceeds 2 seconds: "Time Limit Exceeded" (TLE).
 - If stdout does not match expected output: "Wrong Answer" (WA).
10. **Final Verdict:** If all test cases pass without breaking the loop, the database updates to

“Accepted”. Container is destroyed.

5. Frontend State & UI Flow

5.1 UI Screens

- **Page 1: Authentication Space** (Registration & Login).
- **Page 2: Dashboard** (List of problems).
- **Page 3: Coding Arena** (Problem statement, expandable sample test cases, language dropdown, code editor, custom testcase toggle, *Run Code* button, *Submit Code* button).
- **Page 4: Analytics & Leaderboard** (3 Tabs: My Submissions, All Submissions, Leaderboard).

5.2 Component-Scoped Polling Strategy

- When a user clicks "Submit", React updates the UI to show a loading state and begins polling `/api/submissions/:id` every 2 seconds.
- If the user navigates away from the Coding Arena, the component unmounts, automatically clearing the polling interval to save network resources.
- Users can view finished verdicts seamlessly via the Leaderboard/Submissions tabs.

6: Authentication & Authorization

Role-Based Access Control (RBAC)

- **Standard User Role:** Can read available problems, submit code to the execution engine, view their personal submission history, and view the global feed of submissions for specific problems.
- **Admin Role:** Granted full CRUD (Create, Read, Update, Delete) access to the system. Admins can create new problems, modify hidden test cases, delete faulty submissions, and view the progress of any user on the platform. Protected middleware explicitly verifies the “role” claim in the JWT before allowing access to Admin API routes.

7. Non-Functional Requirements (NFRs)

7.1 Resource Constraints on Docker

- **Memory Limit:** 256 MB.
- **Time Limit:** 2 Seconds.

7.2 Security Measures

7.2.1 JWT

Upon successful login, the Express backend generates a JSON Web Token (JWT) and attaches it to an **HTTP-Only Cookie**. This ensures maximum security by preventing malicious client-side JavaScript (Cross-Site Scripting / XSS) from accessing the token. The browser automatically attaches this cookie to subsequent API requests.

7.2.2 Rate Limiting

To prevent abuse of the Docker execution engine, rate limiting is implemented on two fronts:

- **Frontend UX Constraint:** When a user clicks "Submit", the button immediately enters a disabled, loading state until a verdict is returned, preventing accidental double-clicks or UI spamming.
- **Backend API Gateway:** A rate-limiting middleware strictly throttles the `/api/submissions/submit` endpoint, restricting users to a maximum of 1 submission per 10 seconds per IP address to safeguard the BullMQ queue from malicious scripts.

7.3 Failure Handling

To ensure the Online Judge remains stable during peak traffic and unexpected system faults, the execution pipeline incorporates the following fallback mechanisms:

- **Queue Overload (Surge Handling):** BullMQ inherently protects the backend from being overwhelmed by decoupling submission requests from execution. If submissions exceed the processing rate, they safely stack in the Redis queue.
- **Worker Crashes (Stalled Jobs):** If a Node.js worker abruptly crashes while evaluating a submission, BullMQ's heartbeat monitor will detect the dropped connection. The system marks this as a "stalled job" and automatically returns it to the pending queue. The queue is configured to retry stalled jobs a maximum of 3 times. If it fails on the final attempt, the job is killed, and the database is updated to "Internal System Error."
- **Docker Startup Failures:** If the Docker daemon fails to allocate a container due to host resource exhaustion, the worker pipeline catches this exception via a strict try-catch block. Instead of crashing the worker process, it securely updates the PostgreSQL

database verdict to **Internal System Error**, ensuring the UI can still inform the user.

7.4 Logging Strategy

For the V1 MVP, the backend utilizes standard, efficient local logging:

- **Morgan:** Middleware used to log all incoming HTTP requests for API debugging.
- **Winston:** Used to capture backend exceptions, worker crashes, and database connection failures, writing them to a local log file for easy diagnosis.

8. Deployment

The system is deployed using a decoupled, cloud-native architecture to ensure seamless horizontal scaling in the future:

- **Frontend:** Hosted on a global CDN platform like **Vercel** or **Netlify** for instant asset delivery.
- **Backend, Worker & Queue:** The Node.js Express server, BullMQ Worker, and Docker Engine are hosted together on an **AWS EC2 Instance**.
- **Database:** PostgreSQL is hosted on a managed database service like **AWS RDS** (Relational Database Service) or **Supabase**, completely decoupling the storage layer from the compute layer.

9. Scalability

While V1 prioritizes a solid, functional MVP architecture, the system is designed to accommodate the following scaling upgrades in future iterations:

- **Execution Layer (Horizontal Scaling):** To manage severe overloads in the future, we can configure a concurrency limit on the workers and scale out by spinning up additional worker processes on separate servers that all listen to the same centralized Redis queue.
- **Storage Layer (Object Storage):** While PostgreSQL TEXT columns are great for V1, storing millions of code snippets can eventually bloat the database. In the future, we will migrate the raw submitted code into AWS S3 (Object Storage) to keep the core relational database incredibly lightweight. The database will simply store the S3 file URLs.

10. Current V2 Roadmap (AI Interviewer Integration)

Version 2 will upgrade the Coding Arena into a real-time, AI-driven mock interview environment. The current V1 architecture is designed to seamlessly scale to support the following upgrades without requiring major restructuring/migrations:

10.1 Database Additions (PostgreSQL)

The schema will be updated to handle unstructured AI logic and structured scoring:

- **v2_ai_metadata (Column):** A JSONB column will be added to the “Problems” table to store hidden constraints, specific hints, and edge-case triggers.
- **Interviews (Table):** A new table dedicated to storing the final mock interview sessions to prevent polluting the core “Submissions” table.
 - **Primary Key:**
`interview_id`
 - **Foreign Keys:**
`user_id`,
`problem_id`, and
`Submission_id` (pointing to the user's final submitted (or auto-submitted) code).
 - **Content:**
`chat_transcript` (JSONB).
 - **Scoring:**
`score_breakdown` (JSONB),
`total_score` (INT),
`hunter_rank` (VARCHAR).
- **New Indexes:**
`(user_id)`,
`(problem_id)`,
`(user_id, problem_id)`.

10.2 Additional REST APIs

Endpoint	HTTP Method	Purpose
/api/interviews/start	POST	Initializes the Redis session, sets the 45-minute expiration timer, and returns a WebSocket room ID for the

Endpoint	HTTP Method	Purpose
		frontend to connect to.
/api/interviews/history/me	GET	Fetches the logged-in user's past interview sessions, including their Ranks per submission and full chat transcripts, to populate their Analytics dashboard.
/api/interviews/leaderboard/:problem_id	GET	Fetches the gamified tier list (S-Rank, A-Rank, etc.) for a specific problem.

10.3 Real-Time Communication & State

- **WebSockets:** The Coding Arena will utilize WebSockets (Socket.io) to establish a persistent two-way connection. This allows the frontend to stream IDE changes to the backend in real-time.
- **Redis Session State:** To keep the Node.js servers stateless, the active 45-minute interview state (timer, chat history, hint counters) will be temporarily stored in Redis memory to prevent database lag during the live session.

10.4 AI Scoring Engine & Gamification

When the timer expires, the Node.js backend retrieves the chat history from Redis and the final code from the IDE. It sends this payload to the LLM (e.g., OpenAI/Gemini API) with a strict grading prompt (evaluating speed, hint independence, edge cases, and follow-ups). The LLM returns a comprehensive “score_breakdown” JSON object and a percentage “total_score” (0-100).

The percentage score is mapped to a gamified **Rank** (E-Rank, D-Rank, C-Rank, B-Rank, A-Rank, S-Rank). The user's analytics dashboard will exclusively display this gamified Tier to maintain the immersive experience, while the underlying numerical “total_score” is kept hidden in the database and utilized solely for precise tie-breaking on the global problem leaderboards.

10.5 New UI/UX Workflows

- **The Coding Arena (Mock Interview Mode):** The UI will be updated to feature a side-by-side layout. One half will contain the code editor, and the other half will house the AI Chat Interface and a live 45-minute countdown timer.
- **Gamified Submission Analytics:** The Analytics page will be updated so that each individual mock interview submission displays its awarded **Rank** (e.g., S-Rank, A-Rank). Users do not have a global rank. Rather, each coding attempt is evaluated and ranked independently.
- **The AI Leaderboard:** The new leaderboard tab will rank users not just by "Accepted" code, but by their overall interview performance tiers for specific problems.